# P2474R0
# `views::repeat`

Published Proposal, 2021-12-13

**Author:**
Michał Dominiak

**Project:**
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

**Audience:**
SG9, LEWG

---

## Table of Contents

# § 1. Introduction

This paper proposes two new range factories:

1. `views::repeat`, which is an infinite range that produces the same value repeatedly; and
2. `views::repeat_n`, which is a finite range of a specified size that produces the same value repeatedly.

# § 2. Changelog

## § 2.1. R0

Initial revision.

# § 3. Motivation

// TODO

# § 4. Design

This paper proposes the following additions to the Ranges library:

1. A new view type, `repeat_view`, a view which repeatedly produces the same value, based on the design of `iota_view`.
2. A new customization point object, `views::repeat`, which returns a `repeat_view`, based on the design of `views::iota`.
3. An extension for the semantics of `views::take` and `views::drop`, to make them return bounded `repeat_view` when passed a `repeat_view`.

[P2214R1] suggests that `repeat` and `repeat_n` could be implemented in terms of `generate` and `generate_n`, however it also points out that the implementation of `generate` in range-v3 contains a data race. To avoid this issue, this paper defines `repeat_view` and `repeat_n_view` similarly to how `iota_view` is defined, which avoids the data race in `generate`. (The data race manifests in the implementation of `generate` filling its value cache inside a `const` function. There's a number of ways to resolve this issue, but the author would rather not make solving it a prerequisite for `views::repeat`.)

## § 4.1. `repeat_view`

`repeat_view` is a class template that's parametrized on the type of the value it produces, and an additional parameter which controls whether it is bounded or not. This view is a common range and a sized range when bounded. It is always a random access range.

## § 4.2. `views::repeat`

`views::repeat` is a customization point object which returns an unbounded `repeat_view` when passed just a value, and a bounded `repeat_view` when passed a value and a bound.

## § 4.3. Move-only types and borrowed range

Currently, there is no support for move-only types to be stored inside views; as is, this impacts `views::single` (which requires the value type to be copyable), and the `transform_view` family of adaptors (which requires the transform function to be copyable). The author proposes that this paper follow the requirements in the other adaptors, but also separately proposes to relax these requirements for both the existing adaptors and for the proposed `views::repeat` in P2494.

This paper also proposes that iterators to `repeat_view`s do not store copies of the values. There's several reasons for this. For one, unlike the case of `iota_view`, all the iterators to a `repeat_view` always refer to the same value; copying the values all over to produce the same one from every iterator

does not seem ideal. For another, if the iterators stored copies, the relaxation of the concept requirements mentioned above would not be possible. Finally, there is a precedent for this behavior: `repeat_view` in Range-v3 does not copy the values into iterators, even though it still requires that the values be copyable.

This, however, means that `repeat_view` is **not** a borrowed range. This is a tradeoff, but the author believes that it is a worthy one.

## § 4.4. Category

`repeat_view` is always random access, because the variables being incremented while iterating are always integers or integer-like classes.

## § 4.5. Other properties

`repeat_view` is sized when a bound other than `unreachable_sentinel_t` is provided.

`repeat_view` is common when it is sized.

## § 4.6. `views::take` and `views::drop` changes

`views::take` and `views::drop` are specialized for a sized `iota_view`, in which case they return an `iota_view` back. They require that the `iota_view` be sized, because the bound type for `iota_view` can be an arbitrary type, so it is possible to have an `iota_view` that is random access and finite, but for which there exists no way to obtain the size. In such a case, if we attempted to blindly increment the iterators by the value of the second argument of either `take` or `drop`, we could inadvertently reach an iterator that seems valid, but is in fact past the end of the range.

For `repeat_view`, this situation is much clearer: the value is never modified, and the bound is either `unreachable_sentinel_t`, or an integer like type. In both cases we know that we can reuse the value, and set the bound to the appropriate value. Therefore, this paper proposes to extend the specializations of `views::take` and `views::drop` for `repeat_view`, without requiring that it must be sized.

## § 5. Implementation experience

Range-v3 implements both the bounded and the unbounded variants of `repeat`, but exposes them as separate functions and types. The author believes that this facility should instead follow the design of `views::iota`. The two versions of `views::repeat` in Range-v3 differ only by the existence of the size field in the view itself, by the existence of the `size` function, and by the variant of the `view_facade` base class they inherit from.

## § 6. Wording

### § 6.1. Addition to `<ranges>`

Add the following to 24.2 [range.syn], Header `<ranges>` synopsis:

```
namespace std::ranges {
  // ...

  namespace views { inline constexpr unspecified iota = unspecified; }

  // [range.repeat], repeat view
  template<copy_constructible W, semiregular Bound = unreachable_sentinel_t
  class repeat_view;

  namespace views { inline constexpr unspecified repeat = unspecified; }

  // [range.istream], istream view

  // ...
}
```

### § 6.2. Repeat view [range.repeat]

Add a new section, Repeat view [range.repeat], after Iota view [range.iota].

#### § 6.2.1. Overview [range.repeat.overview]

1. `repeat_view` generates a sequence of elements by repeatedly producing the same value.

2. The name `views::repeat` denotes a customization point object ([customization.point.object]). Given subexpressions `E` and `F`, the expressions `views::repeat(E)` and `views::repeat(E, F)` are expression-equivalent to `repeat_view(E)` and `repeat_view(E, F)`, respectively.

3. [ *Example:*

```
    for (int i : views::repeat(17, 4))
      cout << i << ' '; // prints: 17 17 17 17
```

-- *end example* ]

### § 6.2.2. Class template `repeat_view` [range.repeat.view]

```
namespace std::ranges {
  template<copy_constructible W, semiregular Bound = unreachable_sentinel_t
    requires is-integer-like<Bound> || same_as<Bound, unreachable_sentinel_
  class repeat_view : public view_interface<repeat_view<W, Bound>> {
  private:
    // [range.repeat.iterator], class range_view::iterator
    struct iterator;

    movable-box<W> value_ = W(); // exposition only, see [range.move.wrap]
    Bound bound_ = Bound(); // exposition only

  public:
    repeat_view() requires default_initializable<W> = default;

    template<semiregular Bnd>
    constexpr explicit repeat_view(const repeat_view<V, Bnd> & r,
      type_identity_t<Bound> bound = Bound());
    template<semiregular Bnd>
    constexpr explicit repeat_view(repeat_view<V, Bnd> && r,
      type_identity_t<Bound> bound = Bound());
    constexpr explicit repeat_view(const W & value, Bound bound = Bound());
    constexpr explicit repeat_view(W && value, Bound bound = Bound());
    template<class... Args>
      requires constructible_from<W, Args...>
    constexpr explicit repeat_view(in_place_t, Args &&... args);
    template<class... Args>
      requires constructible_from<W, Args...>
    constexpr repeat_view(in_place_t, Bound bound, Args &&... args);
```

```
    constexpr iterator begin() const;
    constexpr iterator end() const requires !same_as<Bound, unreachable_ser
    constexpr unreachable_sentinel_t end() const;

    constexpr auto size() requires !same_as<Bound, unreachable_sentinel_t>;
  };
}
```

```
template<semiregular Bnd>
constexpr explicit repeat_view(const repeat_view<V, Bnd> & r,
  type_identity_t<Bound> bound = Bound());
```

1. *Effects*: Initializes `value_` with `r.value_` and `bound_` with `bound`.

```
template<semiregular Bnd>
constexpr explicit repeat_view(repeat_view<V, Bnd> && r,
  type_identity_t<Bound> bound = Bound());
```

2. *Effects*: Initializes `value_` with `std::move(r.value_)` and `bound_` with `bound`.

```
constexpr explicit repeat_view(const W & value, Bound bound = Bound());
```

3. *Effects*: Initializes `value_` with `value` and `bound_` with `bound`.

```
constexpr explicit repeat_view(W && value, Bound bound = Bound());
```

4. *Effects*: Initializes `value_` with `std::move(value)` and `bound_` with `bound`.

```
template<typename... Args>
  requires constructible_from<W, Args...>
constexpr explicit repeat_view(in_place_t, Args &&... args);
```

5. *Effects*: Initializes `value_` with `std::forward<Args>(args)...`.

```
template<typename... Args>
  requires constructible_from<W, Args...>
constexpr repeat_view(in_place_t, Bound bound, Args &&... args);
```

6. *Effects*: Initializes `value_` with `std::forward<Args>(args)...` and `bound_` with `bound`.

```
constexpr iterator begin() const;
```

7. *Effects*: Equivalent to `return iterator{&value_};`

```
constexpr iterator end() const requires !same_as<Bound, unreachable_sentine
```

8. *Effects*: Equivalent to `return iterator{&value, bound_};`

```
constexpr unreachable_sentinel_t end() const;
```

9. *Effects*: Equivalent to `return unreachable_sentinel;`

```
constexpr auto size() requires !same_as<Bound, unreachable_sentinel_t>;
```

10. *Effects*: Equivalent to `return bound_;`

### § 6.2.3. Class template `repeat_view::iterator` [range.repeat.iterator]

```
namespace std::ranges {
  template<copyable W, semireular Bound = unreachable_sentinel_t>
    requires is-integer-like<Bound> || same_as<Bound, unreachable_sentinel_
  class repeat_view<W, Bound>::iterator {
  private:
    using index_type = // exposition only
      conditional_t<same_as<Bound, unreachable_sentinel_t>,
        ptrdiff_t,
        Bound
      >;
```

```
    const W * value_; // exposition only
    index_type current_ = index_type(); // exposition only

  public:
    using iterator_concept = random_access_iterator_tag;
    using iterator_catetogy = random_access_iterator_tag;
    using value_type = W;
    using reference = const W &;
    using difference = see below;

    iterator() requires default_initializable<W> = default;

    constexpr explicit iterator(const W * value, index_type b = index_type(

    constexpr const W & operator*() const noexcept;

    constexpr iterator& operator++();
    constexpr iterator operator++(int);

    constexpr iterator& operator--();
    constexpr iterator operator--(int);

    constexpr iterator& operator+=(difference_type n);
    constexpr iterator& operator-=(difference_type n);
    constexpr const W & operator[](difference_type n) const noexcept;

    friend constexpr bool operator==(const iterator& x, const iterator& y);

    friend constexpr bool operator<(const iterator& x, const iterator& y);
    friend constexpr bool operator>(const iterator& x, const iterator& y);
    friend constexpr bool operator<=(const iterator& x, const iterator& y);
    friend constexpr bool operator>=(const iterator& x, const iterator& y);
    friend constexpr auto operator<=>(const iterator& x, const iterator& y)

    friend constexpr iterator operator+(iterator i, difference_type n);
    friend constexpr iterator operator+(difference_type n, iterator i);

    friend constexpr iterator operator-(iterator i, difference_type n);
    friend constexpr difference_type operator-(const iterator& x, const ite
  };
}
```

```
constexpr explicit iterator(const W * value, index_type b = index_type());
```

1. *Effects*: Initializes `value_` with `value` and `bound_` with `b`.

```
constexpr const W & operator*() const noexcept;
```

2. *Effects*: Equivalent to `return *value_;`

```
constexpr iterator& operator++();
```

3. *Effects*: Equivalent to:

```
++current_;
return *this;
```

```
constexpr iterator operator++(int);
```

4. *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
constexpr iterator& operator--();
```

5. *Effects*: Equivalent to:

```
--current_;
return *this;
```

```
constexpr iterator operator--(int);
```

6. *Effects*: Equivalent to:

```
auto tmp = *this;
--*this;
return tmp;
```

```
constexpr iterator& operator+=(difference_type n);
```

7. *Effects*: Equivalent to:

```
current_ += n;
return *this;
```

```
constexpr iterator& operator-=(difference_type n);
```

8. *Effects*: Equivalent to:

```
current_ -= n;
return *this;
```

```
constexpr const W & operator[](difference_type n) const noexcept;
```

9. *Effects*: Equivalent to `return *value_;`

```
friend constexpr bool operator==(const iterator& x, const iterator& y);
```

10. *Effects*: Equivalent to `x.current_ == y.current_;`

```
friend constexpr bool operator<(const iterator& x, const iterator& y);
```

11. *Effects*: Equivalent to `x.current_ < y.current_;`

```
friend constexpr bool operator>(const iterator& x, const iterator& y);
```

12. *Effects*: Equivalent to `x.current_ > y.current_;`

```
friend constexpr bool operator<=(const iterator& x, const iterator& y);
```

13. *Effects*: Equivalent to `x.current_ <= y.current_;`

```
friend constexpr bool operator>=(const iterator& x, const iterator& y);
```

14. *Effects*: Equivalent to `x.current_ >= y.current_;`

```
friend constexpr auto operator<=>(const iterator& x, const iterator& y);
```

15. *Effects*: Equivalent to `x.current <=> y.current_;`

```
friend constexpr iterator operator+(iterator i, difference_type n);
friend constexpr iterator operator+(difference_type n, iterator i);
```

16. *Effects*: Equivalent to `return iterator{i.value_, i.current_ + n};`

```
friend constexpr iterator operator-(iterator i, difference_type n);
```

17. *Effects*: Equivalent to `return iterator{i.value_, i.current_ - n};`

```
friend constexpr difference_type operator-(const iterator& x, const iterato
```

18. *Effects*: Equivalent to `return x.current_ - y.current_;`

## § 6.3. Take view [range.take]

### § 6.3.1. Overview [range.take.overview]

Modify Overview [range.take.overview] paragraph 2 as follows:

2. The name `views::take` denotes a range adaptor object ([range.adaptor.object]). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model

`convertible_to<D>`, `views::take(E, F)` is ill-formed. Otherwise, the expression `views::take(E, F)` is expression-equivalent to:

1. If `T` is a specialization of `ranges::empty_view` ([range.empty.view]), then `((void) F, decay-copy(E))`, except that the evaluations of `E` and `F` are indeterminately sequenced.
2. Otherwise, if `T` models `random_access_range` and `sized_range` and is a specialization of `span` ([views.span]), `basic_string_view` ([string.view]), or `ranges::subrange` ([range.subrange]), then `U(ranges::begin(E), ranges::begin(E) + std::min<D>(ranges::distance(E), F))`, except that `E` is evaluated only once, where `U` is a type determined as follows:
    1. if `T` is a specialization of `span`, then `U` is `span<typename T::element_type>`;
    2. otherwise, if `T` is a specialization of `basic_string_view`, then `U` is `T`;
    3. otherwise, `T` is a specialization of `ranges::subrange`, and `U` is `ranges::subrange<iterator_t<T>>`;
3. Otherwise, if `T` is a specialization of `ranges::iota_view` ([range.iota.view]) that models `random_access_range` and `sized_range`, then `ranges::iota_view(*ranges::begin(E), *(ranges::begin(E) + std::min<D>(ranges::distance(E), F)))`, except that `E` is evaluated only once;
4. Otherwise, if `T` is a specialization of `ranges::repeat_view` ([range.repeat.view]):
    1. if `T` models `sized_range`, then `ranges::repeat_view<range_value_t<T>, D>(std::forward<decltype(E)>(E), std::min<D>(ranges::distance(E), F))`, except that `E` is evaluated only once;
    2. otherwise, `ranges::repeat_view<range_value_t<T>, D>(E, F)`;
5. Otherwise, `ranges::take_view(E, F)`.

## § 6.4. Drop view [range.drop]

### § 6.4.1. Overview [range.drop.overview]

Modify Overview [range.drop.overview] paragraph 2 as follows:

2. The name `views::drop` denotes a range adaptor object ([range.adaptor.object]). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model

`convertible_to<D>`, `views::drop(E, F)` is ill-formed. Otherwise, the expression `views::drop(E, F)` is expression-equivalent to:

1. If `T` is a specialization of `ranges::empty_view` ([range.empty.view]), then `((void) F, decay-copy(E))`, except that the evaluations of `E` and `F` are indeterminately sequenced.
2. Otherwise, if `T` models `random_access_range` and `sized_range` and is

    1. a specialization of `span` ([views.span]),
    2. a specialization of `basic_string_view` ([string.view]),
    3. a specialization of `ranges::iota_view` ([range.iota.view]), or
    4. a specialization of `ranges::subrange` ([range.subrange]) where `T::StoreSize` is `false`,

    then `U(ranges::begin(E) + std::min<D>(ranges::distance(E), F), ranges::end(E))`, except that `E` is evaluated only once, where `U` is `span<typename T::element_type>` if `T` is a specialization of `span` and `T` otherwise.
3. Otherwise, if `T` is a specialization of `ranges::subrange` ([range.subrange]) that models `random_access_range` and `sized_range`, then `T(ranges::begin(E) + std::min<D>(ranges::distance(E), F), ranges::end(E), to-unsigned-like(ranges::distance(E) - std::min<D>(ranges::distance(E), F)))`, except that `E` and `F` are each evaluated only once.
4. Otherwise, if `T` is a specialization of `ranges::repeat_view` ([range.repeat.view]):

    1. if `T` models `sized_range`, then `ranges::repeat_view<range_value_t<T>, D>(std::forward<decltype(E)>(E), ranges::distance(E) - std::min<D>(ranges::distance(E), F));`
    2. otherwise, `E`.
5. Otherwise, `ranges::drop_view(E, F)`.

## § 6.5. Feature-test macro

# § References

## § Informative References

**[P2214R1]**

Barry Revzin, Conor Hoekstra, Tim Song. A Plan for C++23 Ranges. 14 September 2021. URL: https://wg21.link/p2214r1